Intermediate Machine Learning: Assignment 5

Deadline

Assignment 5 is due Thursday, December 4 by 11:59pm. Late work will not be accepted as per the course policies (see the Syllabus and Course policies on Canvas).

Directly sharing answers is not okay, but discussing problems with the course staff or with other students is encouraged. Acknowledge any use of an AI system such as ChatGPT or Copilot.

You should start early so that you have time to get help if you're stuck. The drop-in office hours schedule can be found on Canvas. You can also post questions or start discussions on Ed Discussion. The assignment may look long at first glance, but the problems are broken up into steps that should help you to make steady progress.

Submission

Submit your assignment as a pdf file on Gradescope, and as a notebook (.ipynb) on Canvas. You can access Gradescope through Canvas on the left-side of the class home page. The problems in each homework assignment are numbered. Note: When submitting on Gradescope, please select the correct pages of your pdf that correspond to each problem. This will allow graders to more easily find your complete solution to each problem.

To produce the .pdf, please convert to html and then print to pdf. (You may want to use your pdf print menu to scale the pages to be sure that cells are not truncated.) To convert to html, you can use this converter notebook.

You should start early so that you have time to get help if you're stuck. The drop-in office hours schedule can be found on Canvas. You can also post questions or start discussions on Ed Discussion. The assignment may look long at first glance, but the problems are broken up into steps that should help you to make steady progress.

Topics

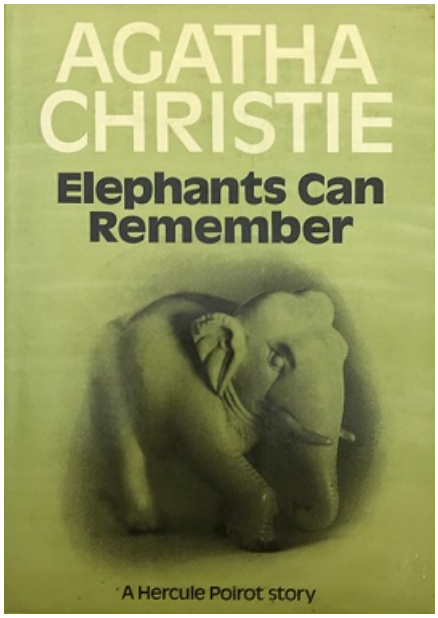
- RNNs and GRUs
- Transformers

After doing this assignment you will have a working knowledge of RNNs and Transformers, and more solid Python skills.

Help Acknowledgement

I used GPT-5.1 for help with this assignment, both with coding as well as the problem solving sections. I also benefitted greatly from discussions with my classmate Jacey.

Problem 1: Elephants Can Remember (25 points)



In this problem, we will work with "vanilla" Recurrent Neural Networks (RNNs) and Recurrent Neural Networks with Gated Recurrent Units (GRUs). The models in this part of the assignment will be character-based models, trained on an extract of the book Elephants Can Remember by Agatha Christie. To reduce the size of our vocabulary, the text is pre-processed by converting the letters to lower case and removing numbers. The code below shows some information about our training and test set. All the necessary files for this problem are available on github.

In [245…
```
import numpy as np
from tqdm import tqdm
import tensorflow as tf
import random
import json

# Ensure tf.keras is the main keras module
# import tensorflow.keras as keras # This line is no longer strictly necessary if all components ar

from tensorflow.keras.models import Sequential, model_from_json
from tensorflow.keras.layers import Dense, Activation
from tensorflow.keras.layers import GRU, SimpleRNN
```

In [246…
```
from google.colab import drive
drive.mount('/content/drive')
data_path = '/content/drive/MyDrive/assn5/'
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/conten
t/drive", force_remount=True).
```

In [247…
```
with open(data_path + 'gru_problem/Agatha_Christie_train.txt', 'r') as file:
    train_text = file.read()

with open(data_path + 'gru_problem/Agatha_Christie_test.txt', 'r') as file:
    test_text = file.read()

vocabulary = sorted(list(set(train_text + test_text)))
vocab_size = len(vocabulary)

# Dictionaries to go from a character to index and vice versa
char_to_indices = dict((c, i) for i, c in enumerate(vocabulary))
indices_to_char = dict((i, c) for i, c in enumerate(vocabulary))
```

In [248…
```
# The first 500 characters of our training set
train_text[0:500]
```

Out[248…
```
'mrs. oliver looked at herself in the glass. she gave a brief, sideways look towards the clock on the mantelpiece, which she had some idea was twenty minutes slow. then she resumed her study of her coiffure. the trouble with mrs. oliver was--and she admitted it freely--that her styles of hairdressing were always being changed. she had tried almost everything in turn. a severe pompadour at one time, then a wind-swept style where you brushed back your locks to display an intellectual brow, at least'
```

In [249…
```
print("The vocabulary contains", vocab_size, "characters")
print("The training set contains", len(train_text) ,"characters")
print("The test set contains", len(test_text) ,"characters")
```

```
The vocabulary contains 44 characters
The training set contains 262174 characters
The test set contains 7209 characters
```

Problem 1.1: The Diversity of Language Models

Before jumping into coding, let's start with comparing the language models we will be using in this assigment.

1. Describe the differences between a Vanilla RNN and a GRU network. In your explanation, make sure you mention the issues with vanilla RNNs and how GRUs try to solve them.

1.1.1 Response

Vanilla RNNs use hidden states to predict words/characters/tokens. This state is updated at each time point using some weighting of the previous state in addition to the current input/previous output. Because the same weight matrix is used across time, one major problem with vanilla RNNs is that the gradients can shrink or explode. Relatedly, there is no explicit way to control what the RNN ``sees'' in memory, since everything is carried in the single hidden state across periods. As a result of both of these mechanisms, vanilla RNNs have a difficult time learning long-range dependencies; for example, a vanilla RNN would struggle to handle sentences where the subject is far away from its verb.

A GRU network, in contrast, is designed to explicitly deal with these issues by introducing two ``gates'' to control the flow of information. An update gate controls how much of the past state to retain from period to period, and a reset gate controls when to ignore past information when transitioning to a new state. Thus, with relatively few additional parameters, we get much better gradient flow and memory control.

2. Describe at least two advantages of a character based language model over a word based language model.

1.1.2 Response

1. **Smaller vocabulary.** Clearly, character based language models require less tokens than word based models (although they will have higher variation in token combinations). Whereas word vocabularies might include thousands of tokens, character vocabularies typically have less than 100 (letters, space, puncuation).
2. **Avoid Out-of-Vocabulary problem.** Word based models can only function on words that the model knows. Character model vocabularies, on the other hand, theoretically handle every word that can be constructed with the characters in the model (in this case, any English word).

Problem 1.2: Generating Text with the Vanilla RNN

The code below loads in a pretrained vanilla RNN model with two layers. The model is set up exactly like in the lecture slides (with tanh activation layers in the recurrent layers) with the addition of biases (intercepts) in every layer (i.e. the recurrent layer and the dense layer). The training process consisted of 30 epochs.

In [250…
```
# load json and create model
json_file = open(data_path + 'gru_problem/RNN_model.json', 'r')
```

```
loaded_model_json_str = json_file.read()
json_file.close()

RNN_model = model_from_json(loaded_model_json_str, custom_objects={'Sequential': Sequential, 'Simpl
RNN_model.load_weights(data_path + "gru_problem/RNN_model.h5")
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass a
n `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Inp
ut(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
```

In [251…

```
# load in the weights and show summary
weights_RNN = RNN_model.get_weights()
RNN_model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
Vanilla_RNN_1 (SimpleRNN)	(None, 100, 128)	22,144
Vanilla_RNN_2 (SimpleRNN)	(None, 64)	12,352
Dense_layer (Dense)	(None, 44)	2,860
Softmax_layer (Activation)	(None, 44)	0

Total params: 37,356 (145.92 KB)
Trainable params: 37,356 (145.92 KB)
Non-trainable params: 0 (0.00 B)

Finish the following function that uses a vanilla RNN architecture to generate text, given the weights of the RNN model, a text prompt, and the number of characters to return. The function should be completed by **only using numpy functions**. Use your knowledge of how every weight plays its role in the RNN architecture. Do not worry about the weight extraction part, this is already provided for you. The weight matrix W_{xh1}, for example, denotes the weight matrix to go from the input x to the first hidden state layer h1. The hidden states h_1 and h_2 are initialized to a vector of zeros.

The embedding of each character has to be done by a one-hot encoding, where you will need the dictionaries defined in the introduction to go from a character to an index position.

In [252…

```
def sample_text_RNN(weights, prompt, N):
    '''
    Uses a pretrained RNN to generate text, starting from a prompt,
    only using the weights and numpy commands
            Parameters:
                    weights (list): Weights of the pretrained RNN model
                    prompt (string): Start of generated sentence
                    N (int): Length of output sentence (including prompt)
            Returns:
                    output_sentence (string): Text generated by RNN
    '''
    # Extracting weights and biases
    # Dimensions of matrices are same format as lecture slides

    # First Recurrent Layer
    W_xh1 = weights[0].T
    W_h1h1 = weights[1].T
    b_h1 = np.expand_dims(weights[2], axis=1)

    # Second Recurrent Layer
    W_h1h2 = weights[3].T
    W_h2h2 = weights[4].T
    b_h2 = np.expand_dims(weights[5], axis=1)

    # Linear (dense) layer
    W_h2y = weights[6].T
```

```
    b_y = np.expand_dims(weights[7], axis=1)

    # Initiate the hidden states
    h1 = np.zeros((W_h1h1.shape[0], 1))
    h2 = np.zeros((W_h2h2.shape[0], 1))

    # ----------------------------------------------

    # Your code starts here
    # ----------------------------------------------
    # Helper: one-hot encoding
    def one_hot(char):
        x = np.zeros((vocab_size, 1))
        idx = char_to_indices[char]
        x[idx, 0] = 1.0
        return x

    # Initialize output sentence with the prompt (possibly truncated if N < len(prompt))
    # Warm up the hidden state with all but the last character of the prompt
    for c in prompt[:-1]:
        x = one_hot(c)
        # First RNN layer
        h1 = np.tanh(W_xh1 @ x + W_h1h1 @ h1 + b_h1)
        # Second RNN layer
        h2 = np.tanh(W_h1h2 @ h1 + W_h2h2 @ h2 + b_h2)

    output_sentence = prompt[:min(len(prompt), N)]
    last_char = prompt[-1]

    # How many new characters we still need to generate
    remaining = max(N - len(output_sentence), 0)

    for _ in range(remaining):
        # Input is one-hot encoding of the last character
        x = one_hot(last_char)

        # Forward pass through first RNN layer
        h1 = np.tanh(W_xh1 @ x + W_h1h1 @ h1 + b_h1)

        # Forward pass through second RNN layer
        h2 = np.tanh(W_h1h2 @ h1 + W_h2h2 @ h2 + b_h2)

        # Output layer (logits)
        y = W_h2y @ h2 + b_y  # (V, 1)

        # Softmax to get probabilities
        logits = y - np.max(y)            # for numerical stability
        exp_logits = np.exp(logits)
        probs = exp_logits / np.sum(exp_logits)

        # Sample next character index from the distribution
        idx_next = np.random.choice(vocab_size, p=probs.ravel())
        next_char = indices_to_char[idx_next]

        # Append to sentence and update last_char
        output_sentence += next_char
        last_char = next_char

    return output_sentence
```

Test out your function by running the following code cell. Use it as a sanity check that your code is working. The generated text should not be perfect English, but at least you should be able to recognize some words.

In [253…

```
print(sample_text_RNN(weights_RNN,
                      'mrs. oliver looked at herself in the glass. she gave a brief, sideways look'
                      1000))

print(sample_text_RNN(weights_RNN,
                      'hercules poirot adjusted his tie carefully and glanced',
```

```
                    1000))
```

```
mrs. oliver looked at herself in the glass. she gave a brief, sideways looked at asking of. they we
re toy the oun will that," said poirot? but it. but say was certainistor of very wogats them to an
y--i upsalle. 't her--was and said know, what there were didn't lad dead," said mrs. oliver, why we
re fai't sain, i she know that well, i dive rempenculty to gat it refor there." "been this happet,"
saig now dons, poores of who look first been profatile a get. sunce,"sen she takeed on the mintey,
but got very did rime." "oh, yes," said mrs. oliver. "i just ter is nost of. i think that was. and
you might aland at has fongre sif the sore one oface. there wasce. any out. "and i'm sure theally m
yst or sfoplening, but whop chackednpigien." "i know excomay." "well, "she was brougany. they'll a
firmn stalaccfaid, looked youn a suster dicishapps aulusthaccef--she was greens alde, that you was
a wor we why, seecture nove the of her mother ow" her so you?" "yes. there got they lund. them buam
ebnt. she
hercules poirot adjusted his tie carefully and glanced to be some apts arreally, i mone for all did
n't know. fon theck. it ore talk at the laver name from learing and comeable. yes. ladry had surpli
ce," shougen for longer sempening to layny say, himly, chan i didn't know you want to iad something
thinks. it wates. it was methat then shouk. i tean erm. "i don't know whes in the were nowad:?" "an
d questinf." "ane she' go a thisk it was there was lack rimens ereppien. you draining. and you when
ching an i winat you mean revent tever. she was, no meak thus, she--she tagh ther girlss, then eed
like things to but she bit of could and she krosentely, i thing a to yeven it leand taides. they th
ere were by his soy, ithes has quite that the lady or give and stonsives. you wele think, and puopn
er leappenal one that ray mrsow, ohe taknowand leaps erseardy." "oh!" "isked to look dealing to tty
time pastearding." it was not the iddad," said mrs. oliver. "is was justhe or ore cixscle, that sai
n quite
```

Problem 1.3: Generating Text with the GRU

The code below loads in a pretrained GRU model. The model is set up exactly like in the lecture slides (with sigmoid activation layers for the gates and tanh activation layers in the recurrent layer). The model is trained for only 10 epochs.

In [254…
```
# load json and create model
json_file = open(data_path + 'gru_problem/GRU_model.json', 'r')
loaded_model_json_str = json_file.read()
json_file.close()

GRU_model = model_from_json(loaded_model_json_str, custom_objects={'Sequential': Sequential, 'GRU':
GRU_model.load_weights(data_path + "gru_problem/GRU_model.h5")
```

In [255…
```
# load in the weights and show summary
weights_GRU = GRU_model.get_weights()
GRU_model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
gru (GRU)	(None, 512)	857,088
dense (Dense)	(None, 44)	22,572
activation (Activation)	(None, 44)	0

Total params: 879,660 (3.36 MB)
Trainable params: 879,660 (3.36 MB)
Non-trainable params: 0 (0.00 B)

Finish the following function that uses a GRU architecture to generate text, given the weights of the GRU model, a text prompt, and the number of characters to return. The function should be completed by **only using numpy functions**. Use your knowledge of how every weight plays its role in the GRU architecture. Do not worry about the weight extraction part, this is already provided for you. The hidden state h is initialized to a vector of zeros.

The embedding of each character has to be done by a one-hot encoding, where you will need the dictionaries defined in the introduction to go from a character to an index position.

Note: a slightly different version of the GRU is used, where the candidate state c_t is calculated as:

$$c_t = \tanh\left(W_{hx}x_t \ + \ \Gamma_t^r \odot (W_{hh}h_{t-1}) \ + \ b_h\right)$$

In [256…

```
# Helper function
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def sample_text_GRU(weights, prompt, N):
    '''
    Uses a pretrained GRU to generate text, starting from a prompt,
    only using the weights and numpy commands
            Parameters:
                    weights (list): Weights of the pretrained GRU model
                    prompt (string): Start of generated sentence
                    N (int): Total length of output sentence
            Returns:
                    output_sentence (string): Text generated by GRU
    '''
    # Extracting weights and biases
    # Dimensions of matrices are same format as lecture slides

    # GRU Layer
    W_ux, W_rx, W_hx = np.split(weights[0].T, 3, axis = 0)
    W_uh, W_rh, W_hh = np.split(weights[1].T, 3, axis = 0)

    bias = np.sum(weights[2], axis=0)
    b_u, b_r, b_h = np.split(np.expand_dims(bias, axis=1), 3)

    # Linear (dense) layer
    W_y = weights[3].T
    b_y = np.expand_dims(weights[4], axis=1)

    # Initiate hidden state
    h = np.zeros((W_hh.shape[0], 1))

    # ------------------------------------------------

    # Your code starts here
    # Input and output dimensions inferred from weights
    input_dim = W_ux.shape[1]     # dimension of one-hot input vector
    V_out = W_y.shape[0]          # number of output characters

    # One-hot encoder that matches the GRU input dimension
    def one_hot(char):
        x = np.zeros((input_dim, 1))
        idx = char_to_indices[char]
        x[idx, 0] = 1.0
        return x

    # Warm up hidden state using all but the last character of the prompt
    for c in prompt[:-1]:
        x = one_hot(c)

        # GRU equations (matching the lecture slides):
        # u_t = sigmoid(W_ux x_t + W_uh h_{t-1} + b_u)
        # r_t = sigmoid(W_rx x_t + W_rh h_{t-1} + b_r)
        # h~_t = tanh(W_hx x_t + W_hh (r_t ⊙ h_{t-1}) + b_h)
        # h_t = (1 - u_t) ⊙ h~_t + u_t ⊙ h_{t-1}
        u_t = sigmoid(W_ux @ x + W_uh @ h + b_u)
        r_t = sigmoid(W_rx @ x + W_rh @ h + b_r)
        h_tilde = np.tanh(W_hx @ x + W_hh @ (r_t * h) + b_h)
        h = (1 - u_t) * h_tilde + u_t * h

    # Start the output with the (possibly truncated) prompt
    output_sentence = prompt[:min(len(prompt), N)]
    last_char = prompt[-1]

    remaining = max(N - len(output_sentence), 0)
```

```
    # Generation loop
    for _ in range(remaining):
        x = one_hot(last_char)

        u_t = sigmoid(W_ux @ x + W_uh @ h + b_u)
        r_t = sigmoid(W_rx @ x + W_rh @ h + b_r)
        h_tilde = np.tanh(W_hx @ x + W_hh @ (r_t * h) + b_h)
        h = (1 - u_t) * h_tilde + u_t * h

        # Output logits and softmax over characters
        y = W_y @ h + b_y
        logits = y - np.max(y)
        exp_logits = np.exp(logits)
        probs = exp_logits / np.sum(exp_logits)

        # Sample next character index and map back to a character
        idx_next = np.random.choice(V_out, p=probs.ravel())
        next_char = indices_to_char[idx_next]

        output_sentence += next_char
        last_char = next_char

    return output_sentence
```

Test out your function by running the following code cell. Use it as a sanity check that your code is working. The generated text should not be perfect English, but at least you should be able to recognize some words.

In [257…
```
print(sample_text_GRU(weights_GRU,
                      'mrs. oliver looked at herself in the glass. she gave a brief, sideways look'
                      1000))

print(sample_text_GRU(weights_GRU,
                      'hercules poirot adjusted his tie carefully and glanced',
                      1000))
```

```
mrs. oliver looked at herself in the glass. she gave a brief, sideways look to entthinges. torable
is tome, ille to wast wide rouse. or. quite a pring! frimchede wase it. 'teorl sherey or and the wo
r ter wedestor, thresere ont to know ise ack a d a ort. "ometimein iele. a deeling to tortaik, you
win'd kily oh. howact shou the molive wert what her sters, andeallis expetivo wore a mattor all sho
is on: selloun in e teerabone oh the poirs. oh. heliabe ambin abersurl goolly? "itel the ithe om to
sle." "ther-she to ard nereve the ieat elied at and and neall she'se ore of the ame yife enteressed
e i very dathing. sor, i moot. " haling to sce stok all about. hamt you unenetevertill ories. herea
ben tire pece the it--""—" "a chally ithin ienote s it,"wpact than allogston. belia, nialsetely dow
ith accois soule dight wayh rateer bod you well. the then i meat out cemis. he megstin, and y me be
very toolive ot." "you ar na eablit and ate tall ablog toor a gatted. olively hade a doknow enderst
in toun
hercules poirot adjusted his tie carefully and glanced and aidentis faritee th to allass she would
nowed elis." frsently betticl weats one in quire th and heddet reteing ttor, to ts torelo neat. "al
otireais, trous it's eleptatle tas or t ine persmenf nimuttistone amo i fict rewes. that, "i'm no t
he ouplears fortielie i hi tirl a matees out ookerione to alaressing on will. tobat entwats a wair
—" dohe terestin ilieghing and she walw af ents ho. wey-siststiren alloo. that. "than o angood a go
t trigge, out ourewicaues or twise, aranythin mrs. outtir s ancer and the rease?" "yes," cane ateer
ed and aba toratiecied?" "apsiale peotings, to sonecait " "mot in airereme tore hed to ertee the th
at. surinientelt, ooce it." "itht' t aroceethi help thr? examenthe wire it some ter onereand thing
to kins on and ewharts and case it hed to twrusict. ond a dowo actesse e. he'se teeseresive to the
willy—ho "idwayss on owien somethin hearsa caterstow and it tolio not wo. look, of you and he tire
in it tull
```

Problem 1.4: Can Elephants Remember Better?

Perplexity is a measure to quantify how "good" a language model M is, based on a test (or validation) set. The perplexity on a sequence s of characters a_i of size N is defined as:

$$\text{Perplexity}(M) = M(s)^{(-1/N)} = \{p(a_1, \ldots, a_N)\}^{(-1/N)} = \{p(a_1)\ p(a_2|a_1)\ \ldots\ p(a_N|a_1, \ldots, a_{N-1})\}^{(-1/N)}$$

> The intuition behind this metric is that, if a model assigns a high probability to a test set, it is not surprised to see it (not perplexed by it), which means the model M has a good understanding of how

> the language works. Hence, a good model has, in theory, a lower perplexity. The exponent $(-1/N)$ in the formula is just a normalizing strategy (geometric average), because adding more characters to a test set would otherwise introduce more uncertainty (i.e. larger test sets would have lower probability). So by introducing the geometric average, we have a metric that is independent of the size of the test set.

When calculating the perplexity, it is important to know that taking the product of a bunch of probabilities will most likely lead to a zero value by the computer. To prevent this, make use of a log-transformation:

$$\text{Log-Perplexity}(M) = -\frac{1}{N} log\left\{p(a_1, \ldots, a_N)\right\} = -\frac{1}{N}\{log\, p(a_1) + \ log\, p(a_2|a_1) + \ \ \ldots \ \ + log\, p(a_N|a_1, \ldots, a_N$$

Don't forget to go back to the normal perplexity after this transformation.

1. Before calculating the perplexity of a test sequence, start with comparing the outputs of 2.2 and 2.3. Do you see any differences in the generated text of the Vanilla RNN model and the GRU model? Rerun your functions a couple of times (because of stochasticity) and use different prompts. Briefly discuss why you would expect (or not expect) certain differences.

1.4.1 Response

I ran the function a few times, and I've included the given prompt in addition to one I came up with (through a conversation with Chat GPT). It seems that the Vanilla RNN actually does somewhat better than the GRU at generating plausible english words (or at least, collections of characters that look more like an english words). One reason we might expect the two models to behave differently is because the GRU needs to train more parameters. So, perhaps a reason for the RNN's relative overperformance is that it has less parameters to train, and so can do more with less data. On the other hand, we would expect a (well-trained) GRU to be able to handle more complex grammar. However, it seems that neither model is in the grammar realm yet, and both still do not form coherent words most of the time.

2. Calculate the perplexity of each language model by using test_text, an unseen extract of the book. Choose the prompt as the first m letters of the test set, where m is a parameter that you can choose yourself. You should be able to reuse the majority of your previous code in this calculation. Discuss your results at the end.

In [258…

```
def perplexity_RNN(weights, text, m):
    # --- unpack weights exactly as in sample_text_RNN ---
    W_xh1 = weights[0].T
    W_h1h1 = weights[1].T
    b_h1 = np.expand_dims(weights[2], axis=1)

    W_h1h2 = weights[3].T
    W_h2h2 = weights[4].T
    b_h2 = np.expand_dims(weights[5], axis=1)

    W_h2y = weights[6].T
    b_y = np.expand_dims(weights[7], axis=1)

    # init hidden states
    h1 = np.zeros((W_h1h1.shape[0], 1))
    h2 = np.zeros((W_h2h2.shape[0], 1))

    def one_hot(c):
        x = np.zeros((vocab_size, 1))
        x[char_to_indices[c], 0] = 1.0
        return x

    # warm up on the first (m-1) characters (no loss yet)
    for i in range(m-1):
        x = one_hot(text[i])
        h1 = np.tanh(W_xh1 @ x + W_h1h1 @ h1 + b_h1)
```

```
        h2 = np.tanh(W_h1h2 @ h1 + W_h2h2 @ h2 + b_h2)

    # now start accumulating log probs from position m onward
    log_prob_sum = 0.0
    N = len(text) - m  # number of predicted characters

    for i in range(m-1, len(text) - 1):
        x = one_hot(text[i])          # current char
        target = text[i+1]            # next char we want probability of

        # forward step
        h1 = np.tanh(W_xh1 @ x + W_h1h1 @ h1 + b_h1)
        h2 = np.tanh(W_h1h2 @ h1 + W_h2h2 @ h2 + b_h2)

        y = W_h2y @ h2 + b_y          # logits (V,1)
        logits = y - np.max(y)        # numerical stability
        exp_logits = np.exp(logits)
        probs = exp_logits / np.sum(exp_logits)

        idx_target = char_to_indices[target]
        p_target = probs[idx_target, 0]
        p_target = max(p_target, 1e-12)   # avoid log(0)
        log_prob_sum += np.log(p_target)

    log_perp = - log_prob_sum / N
    perp = np.exp(log_perp)
    return perp
```

In [259…

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def perplexity_GRU(weights, text, m):
    # --- unpack weights exactly as in sample_text_GRU ---
    W_ux, W_rx, W_hx = np.split(weights[0].T, 3, axis=0)  # (H,V)
    W_uh, W_rh, W_hh = np.split(weights[1].T, 3, axis=0)  # (H,H)

    bias = np.sum(weights[2], axis=0)
    b_u, b_r, b_h = np.split(np.expand_dims(bias, axis=1), 3)

    W_y = weights[3].T               # (V,H)
    b_y = np.expand_dims(weights[4], axis=1)

    h = np.zeros((W_hh.shape[0], 1)) # (H,1)

    def one_hot(c):
        x = np.zeros((vocab_size, 1))
        x[char_to_indices[c], 0] = 1.0
        return x

    # warm up on first (m-1) characters
    for i in range(m-1):
        x = one_hot(text[i])
        u_t = sigmoid(W_ux @ x + W_uh @ h + b_u)
        r_t = sigmoid(W_rx @ x + W_rh @ h + b_r)
        h_tilde = np.tanh(W_hx @ x + W_hh @ (r_t * h) + b_h)
        h = (1 - u_t) * h_tilde + u_t * h

    log_prob_sum = 0.0
    N = len(text) - m

    for i in range(m-1, len(text) - 1):
        x = one_hot(text[i])
        target = text[i+1]

        u_t = sigmoid(W_ux @ x + W_uh @ h + b_u)
        r_t = sigmoid(W_rx @ x + W_rh @ h + b_r)
        h_tilde = np.tanh(W_hx @ x + W_hh @ (r_t * h) + b_h)
        h = (1 - u_t) * h_tilde + u_t * h

        y = W_y @ h + b_y
```

```
        logits = y - np.max(y)
        exp_logits = np.exp(logits)
        probs = exp_logits / np.sum(exp_logits)

        idx_target = char_to_indices[target]
        p_target = probs[idx_target, 0]
        p_target = max(p_target, 1e-12)
        log_prob_sum += np.log(p_target)

    log_perp = - log_prob_sum / N
    perp = np.exp(log_perp)
    return perp
```

In [260…
```
m = 100  # length of prompt
perp_rnn = perplexity_RNN(weights_RNN, test_text, m)
perp_gru = perplexity_GRU(weights_GRU, test_text, m)

print("RNN perplexity:", perp_rnn)
print("GRU perplexity:", perp_gru)
```

```
RNN perplexity: 5.666022529750412
GRU perplexity: 7.185901020565314
```

1.4.2 Results

We confirm that the GRU actually underperforms the vanilla RNN in this setting, which was initially surprising to me. I believe that one reason might be that we only trained the GRU for 10 epochs. Since the GRU has more parameters than the RNN, training lightly like this might lead to worse performance for the more complicated GRU model. In a sense, the undertrained GRU is like a more noisy version of the RNN.

3. As seen in part 2 and 3 of this problem, the text generation is not perfect. Describe some possible model improvements that could make the quality of the generated text better.

1.4.3 Response

As a stated above, I think a big reason for the underperformance, especially of the GRU, is that we did not train the model for very long. So, the first thing I would try would be to increase the amount of epochs for training. Then, we might also try to use a stronger/more complex architecture. For example, the GRU could contain more layers. We might use transformers in place of the vanilla RNN, which as we showed in class tends to perform much better (given that we can train on more data).

Problem 2: Be the Bard (35 points)

Transformer models are the current state of the art in many sequence modeling tasks, and the Transformer architecture underlies most Large Language Models (LLMs), including ChatGPT, Llama, Mistral, etc.

In this problem, we will implement a Transformer language model from scratch in `numpy`. You will be provided with the weights of a small, slightly simplified Transformer language model that we trained on the works of Shakespeare. We will walk through implementing each component of the Transformer architecture and ultimately assemble this into a language model that can generate some text in the style of the Bard.

In [261…
```
import numpy as np
import pickle
#!pip install tiktoken
```

```
import tiktoken
import matplotlib.pyplot as plt
import seaborn as sns
import math
```

In [262…
```
d_model = 512
n_layers = 4
n_heads = 8
d_ff = 4 * d_model
vocab_size = 1024
block_size = 128

model_name = f'BardGPT_weights_D{d_model}L{n_layers}H{n_heads}.npy'
transformer_model_weights = np.load(data_path + '/transformer_problem/' + model_name, allow_pickle=
```

In [263…
```
print('Parameters:')
print('-----------' )
for k, v in transformer_model_weights.items():
    print(f'{k}: {v.shape} [{v.dtype}]')
```

```
Parameters:
-----------
word_embedding.weight: (1024, 512) [float32]
layers.0.attention.wq.weight: (512, 512) [float32]
layers.0.attention.wk.weight: (512, 512) [float32]
layers.0.attention.wv.weight: (512, 512) [float32]
layers.0.attention.wo.weight: (512, 512) [float32]
layers.0.attn_norm.weight: (512,) [float32]
layers.0.attn_norm.bias: (512,) [float32]
layers.0.feed_forward.0.weight: (2048, 512) [float32]
layers.0.feed_forward.2.weight: (512, 2048) [float32]
layers.0.ff_norm.weight: (512,) [float32]
layers.0.ff_norm.bias: (512,) [float32]
layers.1.attention.wq.weight: (512, 512) [float32]
layers.1.attention.wk.weight: (512, 512) [float32]
layers.1.attention.wv.weight: (512, 512) [float32]
layers.1.attention.wo.weight: (512, 512) [float32]
layers.1.attn_norm.weight: (512,) [float32]
layers.1.attn_norm.bias: (512,) [float32]
layers.1.feed_forward.0.weight: (2048, 512) [float32]
layers.1.feed_forward.2.weight: (512, 2048) [float32]
layers.1.ff_norm.weight: (512,) [float32]
layers.1.ff_norm.bias: (512,) [float32]
layers.2.attention.wq.weight: (512, 512) [float32]
layers.2.attention.wk.weight: (512, 512) [float32]
layers.2.attention.wv.weight: (512, 512) [float32]
layers.2.attention.wo.weight: (512, 512) [float32]
layers.2.attn_norm.weight: (512,) [float32]
layers.2.attn_norm.bias: (512,) [float32]
layers.2.feed_forward.0.weight: (2048, 512) [float32]
layers.2.feed_forward.2.weight: (512, 2048) [float32]
layers.2.ff_norm.weight: (512,) [float32]
layers.2.ff_norm.bias: (512,) [float32]
layers.3.attention.wq.weight: (512, 512) [float32]
layers.3.attention.wk.weight: (512, 512) [float32]
layers.3.attention.wv.weight: (512, 512) [float32]
layers.3.attention.wo.weight: (512, 512) [float32]
layers.3.attn_norm.weight: (512,) [float32]
layers.3.attn_norm.bias: (512,) [float32]
layers.3.feed_forward.0.weight: (2048, 512) [float32]
layers.3.feed_forward.2.weight: (512, 2048) [float32]
layers.3.ff_norm.weight: (512,) [float32]
layers.3.ff_norm.bias: (512,) [float32]
fc_out.weight: (1024, 512) [float32]
fc_out.bias: (1024,) [float32]
```

Problem 2.1: Describe & Count the model parameters

Part (a): Describe the role of the parameters of the model. What are the weights `wq.weight` , `wk.weight` ,

`wv.weight` , and `wo.weight` ? What are the dimensions of these matrices? What about the role and dimensions of the feedforward weights `feed_forward.0.weight` and `feed_forward.2.weight` ?

You can refer to the descriptions below as well as lecture notes. Note, in particular, in the comments preceding Problem 2.4 that the attention matrices across heads are "packed together" when you read in the parameters in each layer.

Part (b): Write an expression for the total number of parameters in the model, broken down by each component (Embedding Layer, Attention Layers, Feed Forward Layers, Normalization Layers, and final fully connected layer). Confirm that your answer matches the parameter count in the model weights given to you. Consult the list of parameters above and their shape to resolve any ambiguity regarding variations in Transformer architectures. Write down the expression in terms of: vocabulary size V, model dimension D, number of layers L, and feedforward expansion factor F.

2.1.a Response

The weights `wq.weight` , ... are the linear projection matrices inside the self-attention block. So for each transformer layer, `wq.weight` is the query project, `wk.weight` is the key projection, `wv.weight` is the value projection, and `wo.weight` is the output projection used after concatenation. The first three weights are used to compute attention. The final one is the weight on the concatenated attention. Each matrix has dimension DxD (i.e., 512x512 in this case). The feedforward weights `feed_forward.0.weight` and `feed_forward.2.weight` are the two linear layers in the feedforward network within a transformer layer. The first expands the dimension from D (512) to FD (2048), and the second reduces it back down from FD to D.

2.1.b Response

- The embedding layer has VD parameters. [VD]
- The attention projections have four matrices of size D x D. Each layer norm adds D weights and D bias parameters. [2D]
- Each feedforward layer adds FD^2 weight parameters per weight matrix, of which there are two. [$2FD^2$]
- The feedforward layer norm adds $2D$ parameters. [$2D$]
- The final fully connected layer has VD weight parameters plys V bias parameters. [$VD + V$]

In total, we have $2VD + V + L[(4 + 2F)D^2 + 4D]$ parameters.

Plugging in for $V = 1024$, $D = 512$, $L = 4$, and $F = 4$, we get $13,640,704$ parameters, which matches what we see directly below.

In [264…
```
param_count = np.sum([np.prod(v.shape) for k,v in transformer_model_weights.items()])
print(f"# of Parameters: {param_count:,}")
```

```
# of Parameters: 13,640,704
```

Tokenizer

To process text with a neural network model, we need to first tokenize it in order to convert it to a numerical format that the model can understand and process. The tokenizer converts strings into sequences of integer tokens in a fixed vocabulary. There are different ways to do this. For this problem, we trained a custom Byte-Pair Encoding (BPE) tokenizer on Shakespeare text, setting the vocabulary size to 1024. The code below demonstrates how it works.

In [265…
```
# load the BPE tokenizer
with open(data_path + 'transformer_problem/bpe1024_enc_full.pkl', 'rb') as pickle_file:
    enc = pickle.load(pickle_file)
```

In [266…
```
# text to tokenize
text = """What's in a name? that which we call a rose
By any other name would smell as sweet;"""
```

```
print('Original text:')
print(text)
encoded = enc.encode(text)
print()

print('Encoded:')
print(encoded)
print()

print('Tokens: ')
print([enc.decode([idx]) for idx in encoded])
decoded = enc.decode(encoded)
print()

print('Decoded text:')
print(decoded)
```

```
Original text:
What's in a name? that which we call a rose
By any other name would smell as sweet;

Encoded:
[462, 320, 307, 258, 813, 63, 323, 621, 331, 800, 258, 697, 305, 10, 889, 801, 845, 813, 504, 260,
109, 408, 366, 818, 59]

Tokens:
['What', "'s", ' in', ' a', ' name', '?', ' that', ' which', ' we', ' call', ' a', ' ro', 'se',
'\n', 'By', ' any', ' other', ' name', ' would', ' s', 'm', 'ell', ' as', ' sweet', ';']

Decoded text:
What's in a name? that which we call a rose
By any other name would smell as sweet;
```

Problem 2.2: Token Embeddings

After tokenization, we get a sequence of integers that represent the text to processed, with each integer index corresponding to a particular token in the vocabulary (e.g., a word or word-part). The first step in processing this text is to turn it into a vector representation. This is done via a learned embedding look-up table. For each token in the vocabulary $t \in \mathcal{V}$, we learn an embedding $E_t \in \reals^d$. A sequence of tokens $(t_1, \ldots, t_n)$ is transformed to a vector representation by mapping each token to its embedding $(E_{t_1}, \ldots, E_{t_n}) \in \reals^{n \times d}$. This sequence of vectors is what the neural network model ultimately operates over.

In [267…
```
def embed_tokens(tokens, params):
    """
    Embed tokens using the input embeddings.

    Args:
        tokens (np.array): array of token indices, shape (n_tokens,)
        params (dict): dictionary containing the model parameters
    """

    # get needed parameters
    embeddings = params['word_embedding.weight'] # shape (vocab_size, d_model)
    # embeddings is a look up table for embeddings, with rows corresponding to token indices

    # look up embeddings
    # your code here
    embedded_tokens = embeddings[tokens] # shape (n_tokens, d_model)

    return embedded_tokens
```

In [268…
```
embed_tokens(np.array([1, 2, 3]), params=transformer_model_weights)[:3, :5]
```

Out[268…
```
array([[ 1.8202915 ,  0.49289942, -0.08474425, -1.4825039 ,  1.1505613 ],
       [ 0.01529888,  0.4088627 , -1.3310498 ,  1.5467082 ,  0.7893555 ],
       [ 1.2610923 , -0.06854534, -0.3776905 , -0.45263755,  1.1006896 ]],
      dtype=float32)
```

Expected answer:

```
array([[ 1.8202915 ,  0.49289942, -0.08474425, -1.4825039 ,  1.1505613 ],
       [ 0.01529888,  0.4088627 , -1.3310498 ,  1.5467082 ,  0.7893555 ],
       [ 1.2610923 , -0.06854534, -0.3776905 , -0.45263755,  1.1006896 ]],
      dtype=float32)
```

Positional Encoding

Transformer models are by-default permutation-equivariant. That is, they don't understand order or position. To make them understand positional information, we need to encode it directly in the token embeddings. One way to do this is to represent each possible position i with its own position embedding $PE_i \in \mathbb{R}^d$, and to simpthe positional embedding representing the position of the token.

In the original Transformer paper, the authors propose a particular choice for $PE_i \in \reals^d$ based on sines and cosines with frequencies depending on the position i. Since the original proposal, many follow-up works proposed different positional encoding methods aiming to improve performance and length-generalization. In this problem, we'll use the sinusoidal positional encodings of the original Transformer paper.

We provide the code for computing these sinusoidal positional embeddings below. To give some intuition about the structure of the sinusoidal positional embeddings, we also plot a heatmap of the pairwise inner products $\langle PE_i, PE_j \rangle$. We see that that positions that are closer together have more similar positional embeddings, with additional oscillatory behavior on top of that.

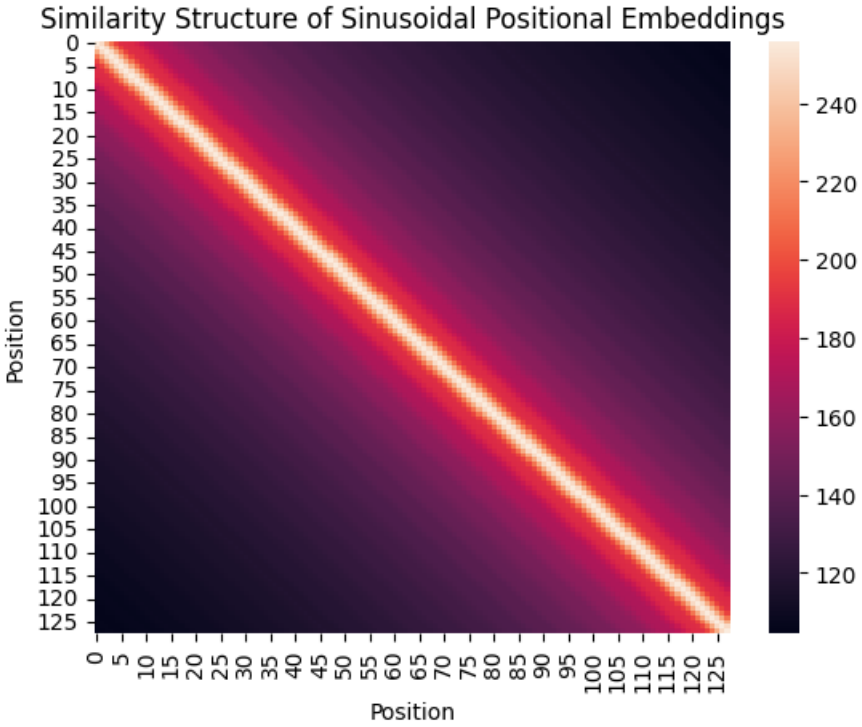
Problem 2.3: Describe the positional embeddings

Below is an implementation of the positional embeddings

In [269…
```python
def get_sinusoidal_positional_embeddings(sequence_length, dim, base=10000):
    inv_freq = 1.0 / (base ** (np.arange(0, dim, 2) / dim))
    t = np.arange(sequence_length)
    sinusoid_inp = np.einsum("i,j->ij", t, inv_freq)

    sin, cos = np.sin(sinusoid_inp), np.cos(sinusoid_inp)

    emb = np.concatenate((sin, cos), axis=-1)
    return emb
```

Explore the positional embeddings by computing the inner-product between all pairs of positional embeddings, and then plotting the similarity matrix as a heat map. Do the results make sense? What are the positional embeddings designed to model? Do they do this effectively? Comment below.

In [270…
```python
pe = get_sinusoidal_positional_embeddings(128, d_model)

#Your code here
pe_similarity = pe @ pe.T
sns.heatmap(pe_similarity)
plt.title('Similarity Structure of Sinusoidal Positional Embeddings')
plt.xlabel('Position')
plt.ylabel('Position')
plt.show()
```



2.3 Response

Yes, it seems like the positional embeddings make sense. We see the strong similarity along the diagonal, so positions are most similar to themselves. Similarity gradually decreases as positions get further apart. These embeddings are designed to measure the absolute position in a sequence. This way, nearby positions have a strong effect on each other, but long-range structure can still be encoded. The gradients will also be smooth, which should help optimization.

Multi-Head Attention

The core operation in a Transformer is multi-head attention, sometimes called self-attention. In this problem, we will implement multi-head attention in numpy from scratch, given the trained parameters of the model.

The input is a sequence of vectors $X = (x_1, \ldots, x_n) \in \reals^{n \times d_{model}}$. For each attention head $h \in [n_h]$, the parameters consist of a query projection matrix $W_q^h \in \reals^{d_{model} \times d_h}$, a key projection matrix $W_k^h \in \reals^{d_{model} \times d_h}$, and a value projection matrix $W_k^h \in \reals^{d_{model} \times d_h}$. Here, d_h is the "head dimension", taken to be d_{model}/n_h (to maintain the same dimensionality between the input and output). The algorithm, for each head, is the following:

1. Compute the queries $Q = XW_q^h$, keys $K = XW_k^h$, and values $V = XW_v^h$. Note that the linear maps are applied independently for each token across the embedding dimension (not sequence dimension), such that $Q, K, V \in \reals^{n \times d_h}$.
2. Compare the queries and keys via inner products to get an $n \times n$ attention matrix $A = \mathrm{Softmax}(QK^\intercal) \in \reals^{n \times n}$.
3. Use the attention scores A to select values, producing the output of the self-attention head: $\mathrm{head}_h = AV \in \reals^{n \times d_h}$. We then concatenate the retrieved values across all heads, and apply a final linear map. Putting this all together yields:

$$\mathrm{head}_h = \mathrm{Softmax}((XW_q^h)(XW_k^h)^\intercal)XW_v^h \tag{1}$$

$$\mathrm{MultiHeadAttention}(X) = \mathrm{concat}(\mathrm{head}_1, \ldots, \mathrm{head}_{n_h})W_o \tag{2}$$

Problem 2.4: Implement multi-head attention

In [271…
```python
# first, implement softmax and causal masking
def softmax(x, axis=-1):

    # your code here
    x_shifted = x - np.max(x, axis=axis, keepdims=True)
    exp_x = np.exp(x_shifted)
    sum_exp = np.sum(exp_x, axis=axis, keepdims=True)
    return exp_x / sum_exp

def apply_presoftmax_causal_mask(attn_scores):
    # apply a causal mask to the attention scores (set the entries above the diagonal to -inf)

    # your code here

    T = attn_scores.shape[-1]

    mask = np.triu(np.ones((T, T), dtype=bool), k=1)

    attn_scores[..., mask] = -np.inf
    return attn_scores
```

In [272…
```python
# check your implementation
x = np.ones((4,4))
softmax_x = softmax(x, axis=1)
print("Softmax:\n", softmax_x)

# apply_presoftmax_causal_mask test
attn_scores = np.ones((4,4))
masked_scores = apply_presoftmax_causal_mask(attn_scores)
print("Masked Attention Scores:\n", masked_scores)

masked_softmax = softmax(masked_scores, axis=-1)
print("Masked Softmax:\n", masked_softmax)
```

```
Softmax:
 [[0.25 0.25 0.25 0.25]
 [0.25 0.25 0.25 0.25]
 [0.25 0.25 0.25 0.25]
 [0.25 0.25 0.25 0.25]]
Masked Attention Scores:
 [[  1. -inf -inf -inf]
 [  1.   1. -inf -inf]
 [  1.   1.   1. -inf]
 [  1.   1.   1.   1.]]
Masked Softmax:
 [[1.         0.         0.         0.        ]
 [0.5        0.5        0.         0.        ]
 [0.33333333 0.33333333 0.33333333 0.        ]
 [0.25       0.25       0.25       0.25      ]]
```

Expected results:

```
Softmax:
 [[0.25 0.25 0.25 0.25]
 [0.25 0.25 0.25 0.25]
 [0.25 0.25 0.25 0.25]
 [0.25 0.25 0.25 0.25]]
Masked Attention Scores:
 [[  1. -inf -inf -inf]
 [  1.   1. -inf -inf]
 [  1.   1.   1. -inf]
 [  1.   1.   1.   1.]]
Masked Softmax:
 [[1.         0.         0.         0.        ]
 [0.5        0.5        0.         0.        ]
 [0.33333333 0.33333333 0.33333333 0.        ]
 [0.25       0.25       0.25       0.25      ]]
```

In [273…

```python
def multi_head_attention(x, params, layer_prefix='layers.0'):
    """
    Compute multi-head self-attention.

    Args:
        x (np.array): input tensor, shape (n, d_model)
        params (dict): dictionary containing the model parameters
        layer_prefix (str): prefix of parameter names corresponding to the layer
        verbose (bool): whether to print intermediate shapes
    """

    # get parameters of multi-head attention layer
    wq = params[f'{layer_prefix}.attention.wq.weight'].T # (d_model, d_model)
    wk = params[f'{layer_prefix}.attention.wk.weight'].T # (d_model, d_model)
    wv = params[f'{layer_prefix}.attention.wv.weight'].T # (d_model, d_model)
    wo = params[f'{layer_prefix}.attention.wo.weight'].T # (d_model, d_model)

    head_dim = d_model // n_heads # dimension of each head
    attn_scale = 1 / math.sqrt(head_dim) # scaling factor for attention scores

    # the wq, wk, wv, wo matrices contain weights for all heads, concatenated
    # first, we split wq, wk, wv, wo into heads
    # note: there are more efficient implementations, but this is more verbose/pedagogical
    wq = wq.reshape(d_model, n_heads, head_dim).transpose(1, 0, 2) # (n_heads, d_model, head_dim)
    wk = wk.reshape(d_model, n_heads, head_dim).transpose(1, 0, 2) # (n_heads, d_model, head_dim)
    wv = wv.reshape(d_model, n_heads, head_dim).transpose(1, 0, 2) # (n_heads, d_model, head_dim)

    head_outputs = []
    for head in range(n_heads):

        # get head-specific parameters (these are the query/key/value projections for this head)
        # your code ehre
        wqh = wq[head] # (d_model, head_dim)
        wkh = wk[head] # (d_model, head_dim)
        wvh = wv[head] # (d_model, head_dim)

        # compute queries, keys, values
        # your code here
        q = x @ wqh # (n, head_dim)
        k = x @ wkh # (n, head_dim)
        v = x @ wvh # (n, head_dim)

        # compute attention scores
        # your code here
        # compute dot product
        attn_scores = q @ k.T # (n, n)
        # multiply by scaling factor
        attn_scores = attn_scores * attn_scale # (n, n)

        # apply causal mask
        attn_scores = apply_presoftmax_causal_mask(attn_scores) # (n, n)
        # apply softmax
        attn_scores = softmax(attn_scores, axis=1) # (n, n)

        # apply attention scores to values
        # your code here
        head_out = attn_scores @ v # (n, head_dim)

        # store the head output
        head_outputs.append(head_out)

    # concatenate all head outputs
    head_outputs = np.concatenate(head_outputs, axis=-1) # (n, d_model)

    # apply output linear map W_o to concatenated head outputs
    # your code here
    output = head_outputs @ wo # (n, d_model)

    return output
```

Test your Attention implementation

To test if you have the correct implementation, you can run the following test line. We show the expected output if your implementation is correct.

In [274…
```
multi_head_attention(np.ones((block_size, d_model)), transformer_model_weights)[:3, :5]
```

Out[274…
```
array([[-0.61194002, -0.31581021,  0.17525476, -0.03176344,  0.23671877],
       [-0.61194002, -0.31581021,  0.17525476, -0.03176344,  0.23671877],
       [-0.61194002, -0.31581021,  0.17525476, -0.03176344,  0.23671877]])
```

Expected output:

```
array([[-0.61194002, -0.31581021,  0.17525476, -0.03176344,  0.23671877],
       [-0.61194002, -0.31581021,  0.17525476, -0.03176344,  0.23671877],
       [-0.61194002, -0.31581021,  0.17525476, -0.03176344,  0.23671877]])
```

MLP

Each Transformer layer (i.e., block) consists of two operations: 1) (multi-head) self-attention, which enables exchange of information between tokens, and 2) a multi-layer perceptron, which processes each token independently. A Transformer model is essentially just alternating between these two operations. In this problem, we will implement the multi-layer perceptron step. Typically, the MLP at each layer is simply a two-layer (one hidden layer) MLP or Feed Forward Network. In our model, we use a ReLU activation in the hidden layer, though other activations are possible. The same MLP network is applied to each token embedding in the sequence independently.

Given $X = (x_1, \ldots, x_n) \in \reals^{n \times d_{model}}$, we apply the MLP as follows:

$$\mathrm{MLP}(X) = \mathrm{ReLU}(XW_1)W_2$$

Note that we don't use biases for simplicity.

Problem 2.5: Implement the MLP

Next, we need to apply the multi-layer perceptron in each layer. Complete the implementation below.

In [275…
```
def relu(x):
    return np.maximum(x, 0)

def mlp(x, params, layer_prefix='layers.0'):
    # get MLP parameters
    w1 = params[f'{layer_prefix}.feed_forward.0.weight'].T # (d_model, d_ff)
    w2 = params[f'{layer_prefix}.feed_forward.2.weight'].T # (d_ff, d_model)

    # Your code here
    hidden = x @ w1
    hidden = relu(hidden)
    o = hidden @ w2 # (n, d_model)

    return o
```

Test your MLP implementation

To test if you have the correct MLP implementation, you can run the following test line. We show the expected output if your implementation is correct.

In [276…
```
mlp(np.ones((block_size, d_model)), transformer_model_weights)[:3, :5]
```

Out[276…
```
array([[ 0.36650751, -0.02109491,  0.13528369, -0.0459696 ,  0.2035102 ],
       [ 0.36650751, -0.02109491,  0.13528369, -0.0459696 ,  0.2035102 ],
       [ 0.36650751, -0.02109491,  0.13528369, -0.0459696 ,  0.2035102 ]])
```

Expected output:

```
array([[ 0.36650751, -0.02109491,  0.13528369, -0.0459696 ,  0.2035102 ],
       [ 0.36650751, -0.02109491,  0.13528369, -0.0459696 ,  0.2035102 ],
       [ 0.36650751, -0.02109491,  0.13528369, -0.0459696 ,  0.2035102 ]])
```

Problem 2.6: Implement Layer Normalization

Layer Normalization is a technique that normalizes the inputs across the features of a layer to stabilize and accelerate training in deep neural networks. It rescales activations to have zero mean and unit variance *within* each layer, then applies a learned gain and shift. In Transformers, LayerNorm is applied before or after sublayers (like attention and feedforward blocks), stabilizing training and maintaining consistent scaling across tokens and layers.

In this problem, we will implement layer normalization. For a vector of activations $x \in \reals^d$, layer normalization returns the normalized feature vector

$$\text{LayerNorm}(x) = \frac{x - E[X]}{\sqrt{\text{Var}[x] + \epsilon}} * \gamma + \beta,$$

where $\gamma \in \reals^d$ is the weight parameter, and $\beta \in \reals^d$ is the bias parameter. Here, $E[\cdot]$ and $\text{Var}[\cdot]$ indicates the mean and variance over the embedding dimension d_{model}.

In [277…
```python
def layer_norm(x, params, layer_prefix='layers.0', norm_type='attn_norm', norm_eps=1e-5):
    # get layer norm params
    weight = params[f'{layer_prefix}.{norm_type}.weight'] # (d_model,)
    bias = params[f'{layer_prefix}.{norm_type}.bias'] # (d_model,)

    # your code here
    mean = x.mean(axis = -1, keepdims = True)
    var = x.var(axis = -1, keepdims = True)

    normed_x = (x - mean) / np.sqrt(var + norm_eps)

    x_norm = normed_x * weight + bias

    return x_norm
```

Test your LayerNorm implementation

To test if you have the correct LayerNorm implementation, you can run the following test line. We show the expected output if your implementation is correct.

In [278…
```python
layer_norm(np.arange(block_size * d_model).reshape(block_size, d_model), transformer_model_weights,
```

Out[278…
```
array([[-1.76364724, -1.75155978, -1.7455241 , -1.74605486, -1.72934376],
       [-1.76364724, -1.75155978, -1.7455241 , -1.74605486, -1.72934376],
       [-1.76364724, -1.75155978, -1.7455241 , -1.74605486, -1.72934376]])
```

Expected Output:

```
array([[-1.76364724, -1.75155978, -1.7455241 , -1.74605486, -1.72934376],
       [-1.76364724, -1.75155978, -1.7455241 , -1.74605486, -1.72934376],
       [-1.76364724, -1.75155978, -1.7455241 , -1.74605486, -1.72934376]])
```

Problem 2.7: Exploring Normalization in Residual Architectures

Transformers are a type of *residual network*. At each layer, the embeddings of each token are iteratively refined by adding the result of a non-linear learnable transformation (either attention or MLP). In general, a residual architecture has the form

$$x^{(\ell+1)} = x^{(\ell)} + \mathcal{F}(x^{(\ell)}),$$

where $x^{(\ell)}$ is the internal representation at layer ℓ and $\mathcal{F}(\cdot)$ is a learned transformation. You can think of $\mathcal{F}(\cdot)$ as

modeling the *residual* $x^{(\ell)} \mapsto x^{(\ell+1)} - x^{(\ell)}$ rather than needing to directly model the full $x^{(\ell)} \mapsto x^{(\ell+1)}$ map. This improves training stability and mitigates vanishing gradient problems.

There are different ways that normalization can be applied to residual networks. The form above includes no normalization.

In the original Transformer paper, the authors use so-called "post-normalization", where the normalization is applied *after* the residual block:

$$x^{(\ell+1)} = \mathrm{LayerNorm}(x^{(\ell)} + \mathcal{F}(x^{(\ell)})).$$

In modern architectures, it is more common to use "pre-normalization", where the normalization before the learned transformation:

$$x^{(\ell+1)} = x^{(\ell)} + \mathcal{F}(\mathrm{LayerNorm}(x^{(\ell)})).$$

In this problem, we will explore the effects of normalization on the magnitude of the feature vectors at each layer. Complete the code below to implement each form of normalization: 1) no norm, 2) post-norm, 3) pre-norm. Plot the average embedding norm across layers for each form of normalization. Interpret the results. Does this shed light onto why normalization is a useful empirical technique to stabilize and accelerate training?

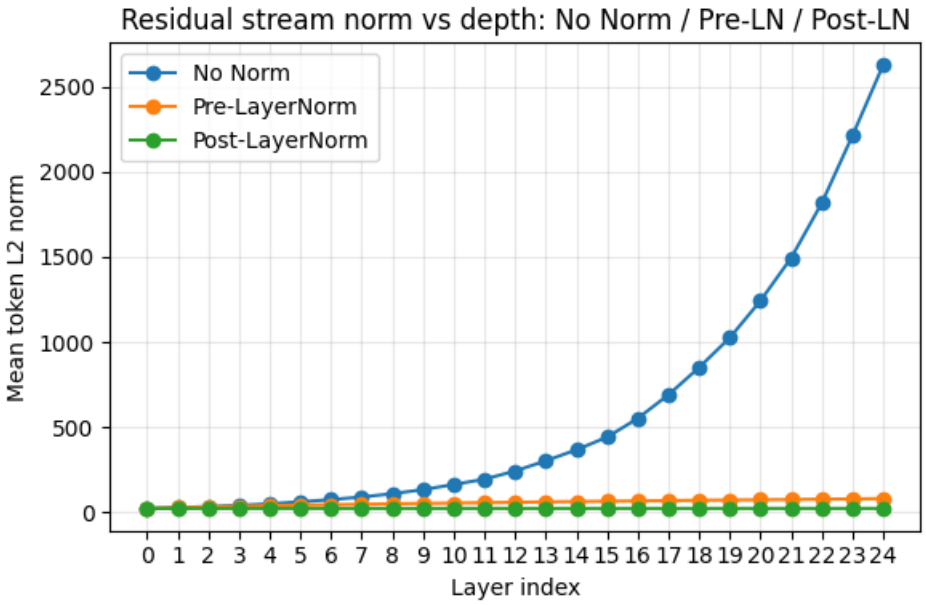
In [279…
```
# Experiment: compare NoNorm, Pre-LN, Post-LN for a stack of feedforward blocks
np.random.seed(0)

seq_len = block_size
d_model = d_model
d_ff = 4 * d_model
n_layers_exp = 24

# shared random input X0
X0 = np.random.randn(seq_len, d_model)

def simple_layer_norm(x, eps=1e-5):
    # for purposes of exploration, implement a simple layer norm with weight = 1 and bias = 0

    # your code here
    mean = x.mean(axis = -1, keepdims = True)
    var = x.var(axis = -1, keepdims = True)

    norm_x = (x - mean) / np.sqrt(var + eps)

    return norm_x

# random FFN blocks to play the role of $\mathcal{F}$
# initialize independent W1/W2 per layer
W1_list = [np.random.randn(d_model, d_ff) * (1.0 / np.sqrt(d_model)) for _ in range(n_layers_exp)]
W2_list = [np.random.randn(d_ff, d_model) * (1.0 / np.sqrt(d_ff)) for _ in range(n_layers_exp)]

def ffn(x, W1, W2):
    return relu(np.dot(x, W1)).dot(W2)
```

In [280…
```
# complete the implementation
def run_scheme(scheme_name):
    x = X0.copy()
    norms = [np.mean(np.linalg.norm(x, axis=1))]  # layer 0
    for i in range(n_layers_exp):
        W1, W2 = W1_list[i], W2_list[i]
        # residual without normalization
        if scheme_name == 'no_norm':
            x = x + ffn(x, W1, W2) # your code here

        # apply "pre-norm" normalization
        elif scheme_name == 'pre_ln':
            x = x + ffn(simple_layer_norm(x), W1, W2) # your code here

        # apply "post-norm" normalization
```

```
        elif scheme_name == 'post_ln':
            x = simple_layer_norm(x + ffn(x, W1, W2)) # your code here
        else:
            raise ValueError(scheme_name)

        # compute norm of each token embedding at current layer
        avg_norm_this_layer = np.mean(np.linalg.norm(x, axis=1)) # your code here
        norms.append(avg_norm_this_layer)
    return np.array(norms)
```

In [281…

```
schemes = ['no_norm', 'pre_ln', 'post_ln']
results = {s: run_scheme(s) for s in schemes}

# Plot
plt.figure(figsize=(6,4))
layers = np.arange(0, n_layers_exp + 1)
plt.plot(layers, results['no_norm'], marker='o', label='No Norm')
plt.plot(layers, results['pre_ln'], marker='o', label='Pre-LayerNorm')
plt.plot(layers, results['post_ln'], marker='o', label='Post-LayerNorm')
plt.xlabel('Layer index')
plt.ylabel('Mean token L2 norm')
plt.title('Residual stream norm vs depth: No Norm / Pre-LN / Post-LN')
plt.xticks(layers)
plt.grid(alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```



2.7 Response

We see that the average embedding norm stays relatively constant for the normalized residuals. On the other hand, the norms explode when we don't apply normalization. Clearly, the normalization helps ensure stability across layers. A bit more subtle, but also important, is the fact that applying the ``post-normalization'' actually causes the norms to shrink, which we also don't want, since it will make optimization more difficult. In the end, the pre-normalization seems to be a good balance, keeping activations stable across layers, allowing for easier optimization and better gradients.

Final Prediction Layer

A Transformer model iteratively applies multi-head attention and MLP layers to process the input. This produces a processed representation of shape $n \times d_{model}$. To make the final prediction (e.g., predict the next token), we need to map the d_{model}-dimensional embedding vectors to logits over the output vocabulary. To do this, we simply apply a

linear map that maps from d_{model} to `vocab_size`.

The starter code for this is given below; you need to complete it.

Problem 2.8: Implement the prediction layer as logits.

In [282…
```
def prediction_head(x, params):
    # get needed parameters
    w = params['fc_out.weight'].T # (d_model, vocab_size)
    b = params['fc_out.bias'] # (vocab_size,)

    # Your code here
    logits = x @ w + b # (n, vocab_size)

    return logits
```

Test the prediction head

In [283…
```
prediction_head(np.ones((block_size, d_model)), transformer_model_weights)[:3, :5]
```

Out[283…
```
array([[-1.52472634, -0.92704511, -0.07956709, -0.27001108, -0.2583226 ],
       [-1.52472634, -0.92704511, -0.07956709, -0.27001108, -0.2583226 ],
       [-1.52472634, -0.92704511, -0.07956709, -0.27001108, -0.2583226 ]])
```

```
array([[-1.52472634, -0.92704511, -0.07956709, -0.27001108, -0.2583226 ],
       [-1.52472634, -0.92704511, -0.07956709, -0.27001108, -0.2583226 ],
       [-1.52472634, -0.92704511, -0.07956709, -0.27001108, -0.2583226 ]])
```

Putting it all together: A Full Transformer Language Model

We are now ready to put this all together to assemble our Transformer Language Model. Recall that the Transformer architecture consists of iteratively applying multi-head attention and MLPs. Each time we apply attention or the MLP, we also apply a *residual connection*: $X^{(\ell+1)} = X^{(\ell)} + F(X^{(\ell)})$. This can be interpreted as a mechanism to enable easy communication between different layers (some people call refer to this idea as the "residual stream"). Real Transformers also include layer normalization in each layer, but we omit this for simplicity in this problem.

The full algorithm is given below:

1. Embed the tokens using the embedding lookup table: $(t_1, \ldots, t_n) \mapsto (E_{t_1}, \ldots, E_{t_n}) =: X^{(0)}$
2. Add the positional embeddings: $X^{(0)} \leftarrow X^{(0)} + (PE_1, \ldots, PE_n)$
3. For each layer $\ell = 1, \ldots, L$:
   A. Apply Multi-Head Attention: $\tilde{X}^{(\ell)} \leftarrow X^{(\ell-1)} + \mathrm{MultiHeadAttention}(\mathrm{LayerNorm}(X^{(\ell-1)}))$.
   B. Apply the MLP: $X^{(\ell)} \leftarrow \tilde{X}^{(\ell)} + \mathrm{MLP}(\mathrm{LayerNorm}(\tilde{X}^{(\ell)}))$.
4. Compute the logits

Problem 2.9: Complete the implementation

Complete the starter code below, which takes embeddings, adds positional encoding, and then adds the attention and MLP components to each layer. Remember that everything is added together, with the computations in one layer added to the outputs of the previous layer, forming the "residual stream".

Hint: to complete the implementation, you will need to use the `layer_norm`, `multi_head_attention`, `mlp`, and `prediction_head` functions you implemented above. Recall that these functions take `param` as input (which are the weights of the pre-trained model). They use the `layer_prefix` argument to fetch the weights at the given layer. For example, `layer_prefix=f'layers.{i}'` will the corresponding module for layer `i`. Additionally, since each layer has two LayerNorms, one for the attention module and one for the MLP module, the `layer_norm` function additionally takes `norm_type` argument, which can be `'attn_norm'` or `'ff_norm'`.

In [284…

```python
def transformer(tokens, params):
    # tokens: (n,) integer array
    # params: dictionary of parameters

    # map tokens to embeddings using embed_tokens
    x = embed_tokens(tokens, params) # (n, d_model)

    # add positional embeddings
    pe = get_sinusoidal_positional_embeddings(x.shape[0], x.shape[1]) # (n, d_model)
    # your code here
    x += pe # (n, d_model)

    # transformer blocks
    for i in range(n_layers):

        layer_prefix = f'layers.{i}'

        # compute multi-head self-attention and add residual with pre-normalization
        # your code here
        normed_x = layer_norm(
            x,params,layer_prefix,norm_type='attn_norm'
        ) # (n, d_model)

        attn_out = multi_head_attention(
            normed_x, params, layer_prefix=layer_prefix
        ) # (n, d_model)

        # residual connection
        x = x + attn_out # (n, d_model)

        # compute MLP and add residual with pre-normalization
        # your code here
        normed_x = layer_norm(
            x,params,layer_prefix,norm_type='ff_norm'
        ) # (n, d_model)

        mlp_out = mlp(
            normed_x, params, layer_prefix=layer_prefix
        ) # (n, d_model)

        # residual connection
        x = x + mlp_out # (n, d_model)

    # compute logits via the prediction_head
    # your code here
    logits = prediction_head(x,params) # (n, vocab_size)

    return logits
```

Test your implementation

You can check your implementation against the expected output below.

In [285…

```python
transformer([0, 1, 2], params=transformer_model_weights)[:3, :5]
```

Out[285…

```
array([[-3.307959 , -1.9419719, -0.6728705, -1.7162718, -1.5415888],
       [-4.1589327, -1.833638 , -2.2626882, -2.63614  , -2.4821224],
       [-3.5148726, -2.3124852, -3.5256255, -3.1805706, -1.7610418]],
      dtype=float32)
```

Expected Output:

```
array([[-3.30795902, -1.94197185, -0.67287023, -1.7162725 , -1.541589  ],
       [-4.15893294, -1.8336369 , -2.26268609, -2.63614073, -2.48212268],
       [-3.51487274, -2.31248397, -3.52562574, -3.18057025, -1.76104193]])
```

Generate some text

Below, we provide some code for generating text from a Transformer language model. The sampling procedure is *autoregressive*. This means that we input some text to the model and it outputs a distribution over next tokens. We sample the next token and append it to the text, then repeat the procedure.

Problem 2.10: Complete the next token generator

Complete the next token generator, but filling in the missing code below. This uses "temperature" to focus on the more probable tokens in a given context (as the temperature decreases).

In [286…

```
def generate_with_transformer(prefix_text, params, max_len=128, greedy=False, temperature=0.9):
    # encode seed text
    prefix_tokens = list(enc.encode(prefix_text))

    # initialize generated tokens
    generated_tokens = prefix_tokens

    # generate new tokens
    for i in range(max_len):
        # predict next token
        logits = transformer(generated_tokens, params)
        # hint: logits[-1] corresponds to prediction of the next token
        if greedy:
            # Your code here
            next_token = int(np.argmax(logits[-1]))
        else:
            # Your code here
            scaled = logits[-1] / temperature

            # stable softmax
            scaled -= np.max(scaled)
            probs = np.exp(scaled)
            probs = probs / np.sum(probs)

            # sample next token
            next_token = int(np.random.choice(vocab_size, p=probs))

        # add next token to generated tokens
        generated_tokens.append(next_token)

    # This converts the tokens to text, using the tiktoken decoder:
    generated_text = enc.decode(generated_tokens)

    return generated_text
```

Test your implementation by generating text. You're the Bard!

Use your implementation to generate text according to the model. Generate text at different temperatures. Do the results make sense? Comment on the quality of the model. What changes to the model would lead to better results? Comment in the Markdown cell below.

In [287…

```
prefix_text = """STUDENT:
O though Transformer, wrought of mind and code,
Thou art the loom where language weaveth thought!
Each token, like a spark of meaning sown,
Attendeth all, and all attend to one."""
```

In [288…

```
generated_text = generate_with_transformer(prefix_text, transformer_model_weights, greedy=True, max
print(generated_text)
```

```
STUDENT:
O though Transformer, wrought of mind and code,
Thou art the loom where language weaveth thought!
Each token, like a spark of meaning sown,
Attendeth all, and all attend to one.



Second Citizen:
Well, sir;
What he's the belly.
Second Citizen:
Well, what then?
First Citizen:
First Citizen:
First Citizen:


If you'll batriciusers, what he hath alre's answer.



If I'll hear, what he hath done fell, what he hath done fell, what answer agused men can bell:

If I'll hear the bell:
If you must not masters
```

In [289…
```
generated_text = generate_with_transformer(prefix_text, transformer_model_weights,
    greedy=False, temperature=0.5, max_len=128)
print(generated_text)
```

```
STUDENT:
O though Transformer, wrought of mind and code,
Thou art the loom where language weaveth thought!
Each token, like a spark of meaning sown,
Attendeth all, and all attend to one.

Second Citizen:
What hearding on would in him worthy.
Well, what we are this deliver.
MENENIUS:
Second Citizen:
Well, sir:
Well, my good friend;
If you'll busers, sir;
I say unto yourselves?
Your most grave belly was accou, my good friend;
Make yourselves?
I say unto the one accunger agude of folly was deliverds, what then?
If
```

In [290…
```
generated_text = generate_with_transformer(prefix_text, transformer_model_weights,
    greedy=False, temperature=1.0, max_len=128)
print(generated_text)
```

```
STUDENT:
O though Transformer, wrought of mind and code,
Thou art the loom where language weaveth thought!
Each token, like a spark of meaning sown,
Attendeth all, and all attend to one.
First Citizen:
If I, barrerong arms are all
therease
And leave me but eale:
You are accusers, guit which aude of you have strong little more.
MARContous rogent to the kill the sick man's all at fire inty.
Your most that trust the diss and proceive the whose court!
ple.
And leave me the discontt, is accusations are must not the point
```

In [291…
```
generated_text = generate_with_transformer(prefix_text, transformer_model_weights,
```

```
    greedy=False, temperature=2.0, max_len=128)
print(generated_text)
```

```
STUDENT:
O though Transformer, wrought of mind and code,
Thou art the loom where language weaveth thought!
Each token, like a spark of meaning sown,
Attendeth all, and all attend to one.To□ my cou fromase his□ stad most that

 well The to ba o' ravise rely you receiveent.irstut callefed other t one honestselkscue ourORraott
er

ide more st□ an I am yselves my goodp□AsT hastp will efeve nat countish some: yetK
□ a□ whoinAndirst prost ofselves doth that hathab siras o'□nway y us to cannot andhars w speak us r
eure;ouradyantyIO v
```

In [292…
```
prefix_text = """HAMLET:
To be, or not to be: that is the question"""
```

In [293…
```
generated_text = generate_with_transformer(prefix_text, transformer_model_weights, greedy=True, max
print(generated_text)
```

```
HAMLET:
To be, or not to be: that is the question,
That, my good friends,'er'd,'er'd,'er
'er came from the luseduselly,'That onceive
And,' What then?

First Citizen:
Should the muniments, inc, ge,

Sir, sir, sir, sir, sir, sir, well.


And you receive the belly's answeral of smeral of Rome are this our own patriciane memeralign our t
ru
```

In [294…
```
generated_text = generate_with_transformer(prefix_text, transformer_model_weights,
    greedy=False, temperature=0.5, max_len=128)
print(generated_text)
```

```
HAMLET:
To be, or not to be: that is the question,
Of the belly,
I' the body, what then?

First Citizen:
MENENIUS:
Ye memer a kind you'll hear the belly,
Stous partic bail, my good friend;
But make my good friend;
That onceive the belly, I do body tale,
First Citizen:
MENENIUS:
Of this most grave belly well;
Not rascraceive
Stenture, what answer a sener made
```

2.10 Response

I generate some text at different temperatures (including the greedy model, which is equivalent to using zero temperature). Overall, the model seems to not only produce english words, but words and phrases in a kind of Shakespearean style, which is awesome. For instance, we see the format matches what you would expect for a play, where the character's name is followed by some dialogue.

It seems to me that the best working model is one that chooses the next word stochastically but with a low temperature, strongly favoring more likely words. For instance, at the highest temperature I tried, we get nonsense that usually aren't even words. On the other end, the greedy generator doesn't produce as convincing text as the

0.5 temperature model, even when the sample text is the Hamlet excerpt.

Unfortunately, we still don't quite get language that makes sense over the course of multiple lines. But it's still cool to see that we can get Shakespear-like language from a toy example like this.

To improve the model, we could try a number of options. Most obviously, we can train the model on more data. Even training on non-Shakespear data might help to improve things like proper english grammar. Probably in addition to more data, we could also make the model itself more complex, adding layers, heads, increasing D, etc. Finally, another complementary option might be to use top-k sampling instead of a full softmax or fully greedy sampling strategy.